

BẢNG PHÂN CÔNG CÔNG VIỆC

BÀI TẬP LỚN LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG— NHÓM 06

Hệ thống đặt vé xem phim trực tuyến — phân công chi tiết

I. Tổng quan phân công

STT	Thành viên	Phụ trách	Vai trò chính
0	Trưởng nhóm	Nền tảng chung + Booking / Ticket + FE	Kiến trúc hệ thống, hạ tầng dùng chung, tích hợp cuối cùng
1	Thành viên 1	Customer & Price Policy	Quản lý khách hàng, chính sách giá vé theo loại khách
2	Thành viên 2	Movie	Quản lý danh sách phim, tìm kiếm phim
3	Thành viên 3	Showtime	Quản lý suất chiếu, liên kết phim & ghế
4	Thành viên 4	Seat + CinemaRoom	Quản lý ghế ngồi, phòng chiếu, đặt ghế
5	Thành viên 5	Payment	Xử lý thanh toán, cập nhật trạng thái vé

II. Phân công chi tiết theo từng thành viên

Mỗi thành viên phụ trách trọn vẹn một module theo mô hình 3 tầng (Model → Repository → Service → Handler, và cung cấp các method công khai (public API nội bộ) để module khác có thể gọi tới.

★. Trưởng nhóm

Nền tảng chung (Giai đoạn 0) + Booking / Ticket

Giai đoạn 0 — bắt buộc hoàn thành trước khi các thành viên khác bắt đầu:

- Khởi tạo Spring Boot project (Java 21+, Maven, Spring Web, Gson), cấu hình CORS, tạo cấu trúc package: model/, repository/, service/, handler/, utils/, exception/, data/.
- Vẽ Class Diagram (9+ class, 2 interface, 1 abstract class, quan hệ extends/implements/association) bằng draw.io hoặc PlantUML.
- Thiết kế schema JSON cho 6 file: customers.json, movies.json, showtimes.json, seats.json, tickets.json, payments.json (đầy đủ field, kiểu dữ liệu).
- Tạo các file JSON rỗng trong thư mục data/.
- Viết utils chung: FileUtils (đọc/ghi JSON), IdGenerator (sinh mã tự tăng), Validator (kiểm tra phone, email, giá).
- Xây dựng exception hierarchy: CustomerNotFoundException, ShowtimeNotFoundException, SeatNotFoundException, SeatAlreadyBookedException, MovieNotShowingException, PaymentFailedException.
- Khai báo interface PaymentMethod và TicketPricePolicy (chưa cài đặt).
- Code Ticket và Booking model (dùng chung TicketRepository).
- Code BookingService với các stub (gọi các method của module khác, tạm trả dữ liệu giả).
- Code BookingHandler (REST endpoint POST /api/bookings).

Giai đoạn cuối:

- Tổng hợp, review code toàn bộ hệ thống và làm FE.
- Viết báo cáo.

✦ Method công khai cho module khác

- TicketRepository.findById()
- TicketRepository.update() (dùng chung cho Booking và Payment)

1. Thành viên 1

Customer & Price Policy

Model:

- NormalCustomer, StudentCustomer, VIPCustomer kế thừa abstract Customer.
- Các enum CustomerType (đã có sẵn từ Giai đoạn 0).

Repository:

- CustomerRepository với các method: findAll(), findById(), save(), update(), deleteById() — sử dụng FileUtils và Gson để đọc/ghi customers.json.

Service:

- CustomerService — CRUD, validate (dùng Validator), tìm theo ID.
- Cài đặt 3 class TicketPricePolicy: NormalCustomerPricePolicy, StudentPricePolicy, VIPPricePolicy — tính giá dựa trên loại khách và loại ghế (VIP +30%, Double +20%, sinh viên -20%, VIP -30%).

Handler:

- CustomerHandler (@RestController): GET /api/customers, GET /api/customers/{id}, POST /api/customers.

✦ Method công khai cho module khác

- CustomerService.findById(String id)
- CustomerService.getPricePolicy(CustomerType type)

2. Thành viên 2

Movie

Model:

- Movie — các field: movieId, title, genre, duration, ageLimit, status (enum MovieStatus).

Repository:

- MovieRepository — findAll(), findById(), findByStatus(), searchByTitle() (không phân biệt hoa thường). Đọc/ghi movies.json.

Service:

- MovieService — lấy danh sách đang chiếu, tìm theo tên, kiểm tra phim có đang chiếu không.

Handler:

- MovieHandler — GET /api/movies (chỉ trả phim đang chiếu), GET /api/movies/search?title=....

✦ Method công khai cho module khác

- MovieService.findById(String id)
- MovieService.isShowing(String movieId)

3. Thành viên 3

Showtime

Model:

- Showtime — các field: showtimeId, movie (Movie), room (CinemaRoom), startTime, endTime, basePrice, danh sách ghế (List<Seat>).

Repository:

- ShowtimeRepository — findAll(), findById(), findByMovieId(). Đọc/ghi showtimes.json.

Service:

- ShowtimeService — kiểm tra suất chiếu tồn tại, thuộc phim đang chiếu, còn ghế trống (gọi SeatService).

Handler:

- ShowtimeHandler — GET /api/showtimes/{id}, GET /api/showtimes?movieId=....

✦ Method công khai cho module khác

- ShowtimeService.findById(String id)
- ShowtimeService.getAvailableSeats(String showtimeId) (gọi xuống SeatService)

4. Thành viên 4

Seat + CinemaRoom

Model:

- Seat — seatId, seatNumber, seatType (enum SeatType), status (enum SeatStatus).
- CinemaRoom — roomId, roomName, totalSeats.

Repository:

- SeatRepository — findAll(), findById(), findByShowtimeId(), findAvailableSeats(), update(Seat). Đọc/ghi seats.json (seed theo showtimeId).
- CinemaRoomRepository — CRUD cơ bản (đọc/ghi cinemareoms.json).

Service:

- SeatService — kiểm tra ghế tồn tại, còn trống, đặt ghế (cập nhật status).
- CinemaRoomService — CRUD đơn giản.

Handler:

- SeatHandler — GET /api/seats/available?showtimeId=....
- CinemaRoomHandler (nếu cần).

✦ Method công khai cho module khác

- SeatService.findById(String id)
- SeatService.isAvailable(String seatId)
- SeatService.bookSeat(String seatId) (cập nhật status)

5. Thành viên 5

Payment

Model:

- Payment — paymentId, ticket (Ticket), method (String), amount, paidAt, success (boolean).

Repository:

- PaymentRepository — findAll(), findById(), save(). Đọc/ghi payments.json.

Service:

- Cài đặt 3 class PaymentMethod: CashPayment, BankTransferPayment, EWalletPayment — xử lý thanh toán (Cash luôn thành công, BankTransfer/EWallet có tỉ lệ thành công 80%).
- PaymentService — nhận ticketId và phương thức, tìm ticket, áp dụng đúng PaymentMethod, cập nhật paymentStatus của Ticket (PAID/FAILED) và lưu Payment.

Handler:

- PaymentHandler — POST /api/payments (body: {ticketId, method}).

✦ Method công khai cho module khác

- Gọi trực tiếp vào TicketRepository.findById() và TicketRepository.update()
- Không gọi qua BookingService

III. Ghi chú chung

- Giai đoạn 0 do Trưởng nhóm hoàn thành trước, làm nền tảng chung cho toàn bộ dự án (model, exception, utils, schema dữ liệu, class diagram).
- Các thành viên chỉ được gọi vào module khác thông qua đúng "Method công khai" đã quy định — không truy cập trực tiếp Repository của module khác.
- Mỗi thành viên chịu trách nhiệm viết Frontend riêng cho module của mình để đảm bảo tiến độ song song.